# PyHelmholtz User Manual

version 0.2.2

PyHelmholtz is a Python package for solving the two-dimensional, time-harmonic Helmholtz equation and comparing absorbing boundary methods (ABMs) in a common finite-difference framework. Its main components are the computational domain, source, absorbing boundary treatment, finite-difference scheme, and solver.

## Features

- Frequency-domain wave propagation modeling.
- Support for standard sparse direct solvers via SciPy.
- Optional high-performance nested dissection algorithms via SuiteSparse.
- Optional parallel direct solver acceleration via MUMPS.

## Installation

PyHelmholtz provides three installation options depending on your performance needs and platform. Python 3.10 or later is required.

### 1. Standard Installation (Recommended)

If you only need standard solvers using SciPy's `spsolve()`, you can install PyHelmholtz instantly with no external C-library or compiler requirements:

```
pip install git+https://github.com/chaiwoot/PyHelmholtz.git@v0.2.2
```

### 2. With Nested Dissection Support (Optional)

To enable advanced nested dissection algorithms, the package requires `scikit-sparse`. Because this depends on underlying SuiteSparse C-libraries, it is highly recommended to install the dependencies via Conda first to avoid compilation issues (especially on Windows):

```
# Install the C-libraries and pre-compiled wrapper via Conda
conda install -c conda-forge scikit-sparse

# Install PyHelmholtz with the optional suitesparse feature flag
pip install "pyhelmholtz[suitesparse] @ git+https://github.com/chaiwoot/
    PyHelmholtz.git@v0.2.2"
```

### 3. With MUMPS Acceleration (Optional, Linux only)

For large-scale problems requiring parallel direct solvers, PyHelmholtz interfaces with MUMPS via PyMUMPS. Because this relies on compiled Fortran, MPI, and BLAS/LAPACK binaries, you must install the system MUMPS libraries first before installing the Python package:

```
# Install system MUMPS and MPI development libraries (Ubuntu/Debian)
sudo apt-get update
sudo apt-get install libmumps-dev

# Install PyHelmholtz with the optional mumps feature flag
```

```
pip install "pyhelmholtz[mumps] @ git+https://github.com/chaiwoot/PyHelmholtz.
    git@v0.2.2"
```

# Quick Start

### Example 1: Default configuration

Here is a quick example of how to use PyHelmholtz to solve the Helmholtz equation using the default parameters. The default parameters are summarized in Table 1.

| Parameters | Values |
|---|---|
| physical domain $\Omega_i$ | [-1, 1] × [-1, 1] $m^2$ |
| velocity $v(x, y)$ | 299792458 m/s (homogeneous) |
| source type | point source |
| source location | (0, 0) m |
| source frequency $f$ | 2 GHz |
| source strength $S$ | 1 |
| grid spacing $h_x$ and $h_y$ | 0.01 m ($h_x = h_y$) |

| Numerical configurations | Settings |
|---|---|
| FD accuracy order | 2 |
| absorbing boundary method | second-order Engquist-Majda (EM2) (one-cell boundary layer) |
| total grid points, $N_x \times N_y$ | 203 × 203 |
| linear solver | SuperLU (`scipy.sparse.linalg.spsolve()`) |

Table 1: Default parameters and numerical configurations of the `Helmholtz` class.

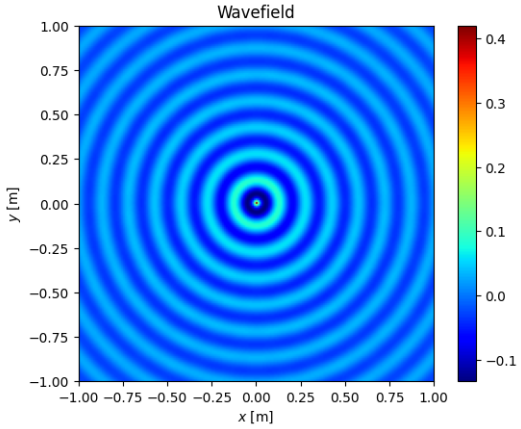



Figure 1: The computed wavefield.

This example uses the package defaults. These defaults define a homogeneous medium with a point source at the origin and the EM2 absorbing boundary treatment. The computed wavefield, as shown in Fig. 1, is stored in the object `ho` and can be accessed using `ho.u`.

```
import pyhelmholtz as ph
ho = ph.Helmholtz() # create a Helmholtz solver with default parameters
ho.solve() # SciPy's spsolve() is used by default as the sparse linear solver
ho.viz() # visualize the real part of the wavefield u
```

**Example 2: Custom parameters 1**

This example defines the domain, source, absorbing boundary method, and finite-difference order separately, then passes them to the main solver.

Here is an example in which several parameters were manually set. These include the grid spacing $h =$ 10 m, the horizontal and vertical domain limits $= [0, 1000] \times [0, 500]$ m$^2$, respectively, the wave speed $=$ 1000 m/s, the point source frequency $=$ 10 Hz and location $=$ [500, 250] m. In addition, the perfectly matched layer (PML) is used as the absorbing boundary method, and the fourth-order FD schemes are used in the calculation.

```
import pyhelmholtz as ph

# Define the computational domain
domain = ph.Domain(h=10, limits=[0, 1000, 0, 500], v=1000)

# Define the point source
source = ph.PointSource(freq=10, xs=500, ys=250)

# Configure the Helmholtz solver
ho = ph.Helmholtz(domain=domain, abm=ph.PML(), source=source, fd=ph.FD(4))

# Solve the Helmholtz equation
ho.solve()

# Visualize the computed wavefield
ho.viz()
```

The results obtained from the code are shown in Fig. 2.

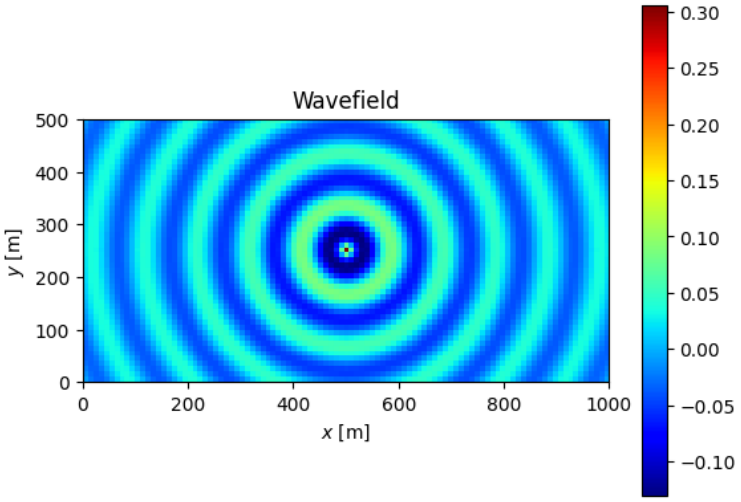



Figure 2: The computed wavefield.

**Example 3: Custom parameters 2**

This example demonstrates how to simulate wave propagation and scattering from a rectangular object using `PyHelmholtz`.

The physical domain is defined by the rectangle [-500, 500] × [-200, 300] m$^2$ and discretized with a grid spacing of $h = 5$ m in both directions. The background wave speed is uniform at 1000 m/s. A rectangular object occupying $[-200, 50] \times [-100, 100]$ m$^2$ is embedded in the background, with a wave speed of 800 m/s. A plane wave with a frequency of 10 Hz propagates through the domain at an incident angle of 0° relative to the positive $x$-axis. A perfectly matched layer (PML) with 15 cells is used to absorb outgoing waves, and a fourth-order finite-difference scheme is used for spatial discretization. The minimum wavelength is $\lambda_{\min} = 80$ m, giving approximately 16 grid points per minimum wavelength ($\lambda_{\min}/h = 16$).

```
import pyhelmholtz as ph
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec

# Define the computational domain
domain = ph.Domain(limits=(-500, 500, -200, 300), h=5, v=1000)
domain.add_rectangle(bounding_box=(-200, 50, -100, 100), v=800)

# Define the planewave source
source = ph.PlaneWave(freq=10, v0=1000, theta=0)

# Configure the Helmholtz solver and solve the Helmholtz equation
ho = ph.Helmholtz(domain=domain, abm=ph.PML(n=15),
                  source=source, fd=ph.FD(order=4))
ho.solve()

# Visualize the results
fig = plt.figure(figsize=(15, 8), constrained_layout=True)
gs = GridSpec(2, 8, figure=fig)
ax1 = fig.add_subplot(gs[0, 2:6])
ax2 = fig.add_subplot(gs[1, 0:4])
ax3 = fig.add_subplot(gs[1, 4:8])
plt.sca(ax1); ho.viz(data="velocity")
plt.sca(ax2); ho.viz(data="solution")
plt.sca(ax3); ho.viz(data="total")
plt.show()
```

The results obtained from the code are presented in Fig. 3.

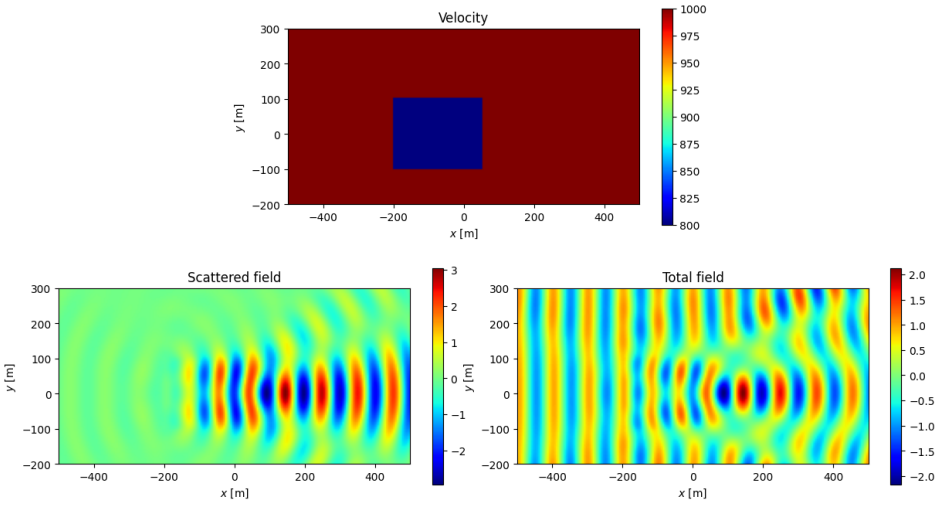



Figure 3: The velocity model, the computed scattered and total wavefields.

# API Reference

## 1. Domain

The `Domain` class defines the 2D physical domain, its spatial grid, and background wave-speed model. It also provides utility methods for embedding geometric velocity anomalies into the grid.

```
ph.Domain(limits=(-1,1,-1,1), h=0.01, v=299792458, positive_downward=False)
```

| Parameter | Data type | Description | Default |
|---|---|---|---|
| `limits` | tuple of float | Physical domain $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ [m] | `(-1, 1, -1, 1)` |
| `h` | float or array-like | Grid spacing in the $x$- and $y$-directions [m] | `0.01` |
| `v` | float or array-like | Background wave speed [m/s];<br>scalar (homogeneous)<br>or spatially varying (inhomogeneous) | `299792458` (light speed) |
| `positive_downward` | bool | If `True`, the $y$-axis points downward | `False` |

The `Domain` class provides two methods for defining objects in the physical domain: `Domain.add_circle()` and `Domain.add_rectangle()`.

The method `Domain.add_circle()` assigns a specified value to grid points inside or on the boundary of a circle.

```
domain = ph.Domain()
domain.add_circle(center, radius, v)
```

| Parameter | Data type | Description | Default | Example |
|---|---|---|---|---|
| `center` | tuple of float | Center $(x_c, y_c)$ [m] | – | `(-1.5, 1.5)` |
| `radius` | float | Radius [m] | – | `3.2` |
| `v` | float | Wave speed (homogeneous) [m/s] | – | `2e8` |

The method `Domain.add_rectangle()` assigns a specified value to grid points within a rectangular region defined by $[x^b_{\min}, x^b_{\max}] \times [y^b_{\min}, y^b_{\max}]$, including the endpoints.

```
domain.add_rectangle(bounding_box, v)
```

| Parameter | Data type | Description | Default | Example |
|---|---|---|---|---|
| `bounding_box` | tuple of float | Bounding box limits<br>$[x^b_{\min}, x^b_{\max}] \times [y^b_{\min}, y^b_{\max}]$ [m] | – | `(-0.3, 0.3, -0.8, -0.2)` |
| `v` | float | Wave speed (homogeneous) [m/s] | – | `2.5e8` |

## 2. Source

The `Source` class defines source models for the 2D frequency-domain Helmholtz equation. `Source` is the abstract base class, and `PointSource` and `PlaneWave` are concrete implementations. These two classes construct the source vector $\mathbf{b}$ of the linear system $\mathbf{Ax} = \mathbf{b}$.

### 2.1 Point source

`PointSource` defines one or more point sources, $S_j\delta(x - x_{s,j}, y - y_{s,j})$, at coordinates $(x_{s,j}, y_{s,j})$ for the specified frequency `freq`, where $S_j$ denotes the strength of the $j$-th source. The source locations and strengths determine the entries of the source vector $\mathbf{b}$.

```
ph.PointSource(freq=2e9, xs=0.0, ys=0.0, strength=1.0)
```

| Parameter | Data type | Description | Default |
|---|---|---|---|
| `freq` | float | Source frequency [Hz] | `2e9` |
| `xs` | float or array-like | $x$-coordinates of the source(s) [m] | `0.0` |
| `ys` | float or array-like | $y$-coordinates of the source(s) [m] | `0.0` |
| `strength` | float or array-like | Source strength(s) [a.u.] | `1.0` |

### 2.2 Plane wave

`PlaneWave` defines an incident plane wave propagating at an angle `theta` (in degrees) relative to the positive $x$-axis. The incident plane wave is expressed as

$$u^{\text{inc}}(x, y) = Se^{ik_x(x-x_{\text{zp}})}e^{ik_y(y-y_{\text{zp}})},$$

where $S$ is the incident wave amplitude, $k_0$ is the background wavenumber, and $k_x = k_0 \cos\theta$ and $k_y = k_0 \sin\theta$ are the $x$- and $y$-components of the background wavenumber vector, respectively. The point $(x_{\text{zp}}, y_{\text{zp}})$ specifies the reference location for the wave phase.

```
ph.PlaneWave(freq=2e9, v0=299792458, strength=1.0, theta=0.0,
             x_zerophase=0.0, y_zerophase=0.0)
```

| Parameter | Data type | Description | Default |
|---|---|---|---|
| `freq` | float | Incident-wave frequency [Hz] | `2e9` |
| `v0` | float | Background wave speed [m/s] | `299792458` (light speed) |
| `strength` | float | Incident-wave amplitude [a.u.] | `1.0` |
| `theta` | float | Propagation angle relative to the positive $x$-axis [degrees] | 0.0 |
| `x_zerophase` | float | $x$-coordinate of the phase reference point [m] | `0.0` |
| `y_zerophase` | float | $y$-coordinate of the phase reference point [m] | `0.0` |

## 3. Absorbing Boundary Method (ABM)

The `ABM` class defines the interface for assembling a system matrix $\mathbf{A}$ with a selected boundary treatment. The implementations comprise `RenLiu` and `PML`.

### 3.1 Ren–Liu hybrid method

The `RenLiu` class supports the Engquist–Majda and Higdon absorbing boundary conditions, as well as Ren–Liu hybrid formulations.

```
# Higdon absorbing boundary method (n=1)
ph.RenLiu(abm="H2")

# Ren-Liu hybrid method with EM1
# Absorbing thickness: 15 cells
ph.RenLiu(abm="RLEM1", n=15)
```

| Parameter | Data type | Description | Default |
|---|---|---|---|
| `abm` | string | Absorbing boundary method | `"EM2"` |
| `n` | integer | Number of absorbing layers | `1` |
| `damping_profile` | callable | Weighting profile for the Ren–Liu method (`n` $\geq 2$) | `lambda i,n: (i/n)**2` |

The supported ABM names for the `RenLiu` class and their allowed layer thicknesses `n` are listed in the table below.

| `abm` | Absorbing boundary method | Allowed `n` |
|---|---|---|
| `"EM1"` | First-order Engquist–Majda | 1 |
| `"EM2"` | Second-order Engquist–Majda (default) | 1 |
| `"H2"` | Second-order Higdon ($\phi = 0°, 45°$) | 1 |
| `"RLEM1"` | Ren–Liu hybrid with first-order Engquist–Majda | $\geq 2$ |
| `"RLEM2"` | Ren–Liu hybrid with second-order Engquist–Majda | $\geq 2$ |
| `"RLH2"` | Ren–Liu hybrid with second-order Higdon | $\geq 2$ |

The parameter `damping_profile` specifies how the weighting coefficient varies across the absorbing layers. It accepts a callable that takes the layer index `i` and the total number of layers `n` as inputs. The default profile is quadratic.

### 3.2 Perfectly Matched Layer (PML)

The `PML` class supports the unsplit-field PML and PML–ABC hybrids.

```
# PML with second-order Engquist--Majda outer boundary
# PML thickness: 15 cells
# Target reflection coefficient: R0 = 2e-6
ph.PML(abm="PMLEM2", n=15, R0=2e-6)
```

| Parameter | Data type | Description | Default |
|---|---|---|---|
| `abm` | string | Absorbing boundary method | `"PML0"` |
| `n` | integer | Number of PML layers (`n` $\geq 2$) | `5` |
| `damping_profile` | callable | Damping profile for the PML method | `lambda i, n: 1/(1+1j*(sigma_max/omega)*(i/(n-1))**m)` |
| `R0` | float | Target normal-incidence reflection coefficient $R_0$ | `1e-5` |

The supported ABM names for the `PML` class and their allowed layer thicknesses `n` are listed in the table below.

| `abm` | Absorbing boundary method | Allowed `n` |
|---|---|---|
| `"PML0"` | PML with Dirichlet termination (default) | $\geq 2$ |
| `"PMLEM1"` | PML with first-order Engquist–Majda outer boundary | $\geq 2$ |
| `"PMLEM2"` | PML with second-order Engquist–Majda outer boundary | $\geq 2$ |
| `"PMLH2"` | PML with second-order Higdon outer boundary | $\geq 2$ |

The parameter `damping_profile` for the `PML` class defines how the complex stretching factor varies across the PML region. It accepts a callable that takes the layer index `i` and the total number of PML layers `n` as inputs.

For the $x$-direction, the complex stretching factor is defined as

$$s_x = 1 + \frac{\mathrm{i}\sigma_{\max}}{\omega}\left(\frac{x}{L}\right)^m,$$

where $\sigma_{\max}$ is the maximum damping coefficient, $\omega$ is the angular frequency, $L$ is the PML thickness, and $m$ controls the damping profile. The default profile is quadratic, with $m = 2$.

The target normal-incidence reflection coefficient $R_0$ and the PML thickness $L$ determine the maximum damping coefficient [1]:

$$\sigma_{\max} = -\frac{(m+1)v\ln R_0}{2L},$$

where $v$ is the wave speed and $L = (n-1)h$ is the physical PML thickness, excluding the outer boundary grid point.

## 4. Finite Difference Order

The `FD` class specifies the finite-difference order used to approximate spatial derivatives on the grid. Higher-order schemes can improve accuracy for sufficiently smooth, well-resolved fields, but require wider stencils and more neighboring grid points.

```
ph.FD(order=4)
```

| Parameter | Data type | Description | Default |
|---|---|---|---|
| `order` | integer | finite-difference accuracy order (2 or 4) | `2` |

## 5. Core Helmholtz Solver

The `Helmholtz` class binds together domain, source, absorbing boundary treatment, and finite-difference scheme. It manages matrix assembly, linear system solving, data visualization, and analytic verification.

```
ph.Helmholtz(domain=ph.Domain(), source=ph.PointSource(), abm=ph.RenLiu(), fd=
    ph.FD())
```

| Parameter | Type | Description |
|---|---|---|
| `domain` | `Domain` | Defines the physical domain and wave-speed model |
| `source` | `PointSource`, `PlaneWave` | Defines the excitation source |
| `abm` | `RenLiu`, `PML` | Defines the absorbing boundary method |
| `fd` | `FD` | Defines the finite-difference discretization |

The `Helmholtz` class provides four main methods: `build()`, `solve()`, `viz()` and `error_norm_point_source()`.

### 5.1 `build`

The method `build` assembles the discrete Helmholtz system matrix $\mathbf{A}$ and source vector $\mathbf{b}$ and returns them as a pair.

```
ho = ph.Helmholtz()

# A: Complex sparse system matrix (CSR format), shape (m, m)
# b: Complex source vector, shape (m,)
A, b = ho.build()
```

### 5.2 `solve`

The method `solve` builds and solves the linear system $\mathbf{Ax} = \mathbf{b}$ using the selected numerical solver. The computed wavefield is stored in the object's `u` attribute.

```
ho = ph.Helmholtz()
ho.solve() # use the default solver (spsolve)
ho.solve(solver="mumps", nprocs=4) # use MUMPS (if installed) and 4 MPI cores
```

| Parameter | Type | Description | Default |
|---|---|---|---|
| `solver` | string | Numerical solver | `"spsolve"` |
| `nprocs` | integer | number of MPI processors for MUMPS | `1` |

The supported values of `solver` are listed in the table below.

| `solver` | Description | Type of solver |
|---|---|---|
| `"spsolve"` | SciPy sparse direct solver | Sparse direct solver |
| `"mumps"` | Parallel sparse solver via PyMUMPS | Parallel sparse direct solver |
| `"nested_dissection"` | Nested-dissection reordering via CHOLMOD | Sparse direct solver |
| `"gmres"` | GMRES with shifted ILU preconditioning | Iterative solver |

### 5.3 `viz`

The method `viz` displays a 2D plot of the computed wavefield or velocity model. It can show the real part, imaginary part, or magnitude of the selected data.

```
ho = ph.Helmholtz()
ho.viz(data="solution", mode="real", unit="m",
       cmap="jet", vlim=None, title="",
       xlabel=True, ylabel=True)
```

The parameters accepted by `viz()` are summarized below.

| Parameter | Type | Description | Available options | Default |
|---|---|---|---|---|
| `data` | string or array | Data to visualize | `"solution"`, `"velocity"`, `"incident"`, `"total"`, NumPy array | `"solution"` |
| `mode` | string | Quantity to display | Real part (`"real"`), Imaginary part (`"imag"`), Magnitude (`"abs"`) | `"real"` |
| `unit` | string | Spatial coordinate unit | `"km"`, `"m"`, `"cm"`, `"mm"`, `"um"`, `"nm"` | `"m"` |
| `cmap` | string | Matplotlib colormap | – | `"jet"` |
| `vlim` | tuple or list | Color scale limits $(v_{\min}, v_{\max})$ | – | `None` |
| `title` | string | Plot title | – | `""` |
| `xlabel` | bool | Display the x-axis label if `True` | – | `True` |
| `ylabel` | bool | Display the y-axis label if `True` | – | `True` |

If `data` is a NumPy array, it must have the same shape as the computed wavefield. The `unit` parameter changes the displayed coordinate scale. It does not change the underlying simulation grid.

### 5.4 `error_norm_point_source`

The method `error_norm_point_source` calculates the relative $L_2$ error, expressed as a percentage, between the numerical point-source solution and the analytical reference field.

```
ho = ph.Helmholtz()
ho.solve()

error_percent = ho.error_norm_point_source()
print(error_percent)
```

This example gives a relative $L_2$ error of 16.3% for the default configuration: the EM2 absorbing boundary method, second-order finite differences, and grid spacing $h = 0.01$ m (approximately 15 grid points per wavelength).

## References


[1] Collino F, Tsogka C. Application of the perfectly matched absorbing layer model to the linear elastodynamic problem in anisotropic heterogeneous media. Geophysics. 2001;66(1):294-307.